

RK3568 AMP架构：裸机异常与崩溃调试机制 (Crash Dump)

文档状态：已归档 **关联文件：** `crash.c` **适用场景：**RK3568 平台，FreeRTOS/裸机环境 (Core1/M3) 的严重硬件异常排查。

1. 设计背景与哲学

在复杂的 AMP（非对称多处理）架构中，从核（ 运行 FreeRTOS 的 Core 1/3）极易因为跨核通信的时序问题、内存映射错误或权限越界引发严重硬件异常（ 总线错误 Data Abort、指令错误 Instruction Abort）。

当系统陷入崩溃时，RTOS 调度器、中断控制器 (GIC) 以及标准的 C 库函数（ `printf` ）通常已经失效或不再安全。因此，我们需要一个“Panic 环境极简设计法则”的调试机制：

- 无堆栈依赖：**极少使用局部变量，防止二次栈溢出。
- 无中断依赖：**纯轮询（Polling）模式操作外设。
- 绝对独立：**绕开所有驱动框架，直接通过物理内存地址读写（MMIO）操作 UART 寄存器。

`crash.c` 正是基于这一哲学实现的“崩溃遗言”打印工具。

2. 核心硬件寄存器解析 (ARMv8 AArch64)

在 `abort_handler` 函数中，我们通过内联汇编提取了 ARMv8 架构下定位崩溃的“四大金刚”寄存器。它们是案发现场最核心的法医鉴定证据。

2.1 提取代码实现

```
uint64_t esr, far, elr, sp;
__asm volatile("mov %0, sp" : "=r"(sp));
__asm volatile("mrs %0, elr_el1" : "=r"(elr));
__asm volatile("mrs %0, esr_el1" : "=r"(esr));
__asm volatile("mrs %0, far_el1" : "=r"(far));
```

2.2 寄存器功能详解

- SP (Stack Pointer - 栈指针)**
 - 作用：**记录异常发生瞬间的栈顶物理地址。可用于导出 Stack Dump，还原函数调用链（Backtrace）。
- ELR_EL1 (Exception Link Register - 异常链接寄存器)**
 - 作用：**即 **PC (Program Counter) 指针**。保存了触发异常的指令地址或异常返回地址。通过 `aarch64-none-elf-addr2line -e firmware.elf <ELR地址>` 可以直接定位到 C 代码的具体行号。
- ESR_EL1 (Exception Syndrome Register - 异常状态寄存器)**
 - 作用：**崩溃的“病理报告”。其 Exception Class (EC) 字段等包含了异常的类型（ Data Abort, EC=0 未知原因同步异常等）。
- FAR_EL1 (Fault Address Register - 故障地址寄存器)**
 - 作用：**记录引发异常的**非法目标内存地址**。例 试图读写未映射的地址、或者是越权访问 TrustZone 保护的地址时，该地址会被捕获于此。

3. 极简轮询串口驱动实现

为了在崩溃后将上述寄存器的值输出给开发者， `crash.c` 实现了一个微型的、直接操作 16550 UART IP 核的串口发送模块。

3.1 物理地址定义

```
#define EXC_UART2    0xFE660000UL // RK3568 UART2 物理基地址
```

3.2 单字符轮询发送 (`exc_putc`)

```
static void exc_putc(char c)
{
    /* 偏移 0x14 为 LSR (线路状态寄存器) */
    volatile uint32_t *lsr = (volatile uint32_t *) (EXC_UART2 + 0x14);
    /* 偏移 0x00 为 THR (发送保持寄存器) */
    volatile uint32_t *thr = (volatile uint32_t *) (EXC_UART2 + 0x00);

    /* 轮询 LSR 第 5 位 (THRE: Transmit Holding Register Empty)
     * 为 0 表示仍在发送，死等；为 1 表示空闲，跳出循环 */
    while (!( *lsr & (1u << 5))) ;

    /* 将字符写入 THR 寄存器，触发硬件发送 */
    *thr = (uint32_t) c;
}
```

3.3 字符串与终端换行适配 (`exc_puts`)

终端显示工具遇到 `\n` 通常只作下移行处理。此函数在遇到 `\n` 时自动前置发送 `\r` (回车)，保证崩溃日志在终端上完美对齐。

```
static void exc_puts(const char *str)
{
    while (*str) {
        if (*str == '\n') exc_putc('\r');
        exc_putc(*str++);
    }
}
```

3.4 64位数值转十六进制字符串 (`exc_hex`)

无法使用 `<stdio.h>` 的 `printf` ，必须手动实现高效率的数值转字符串。

```
static void exc_hex(uint64_t v)
{
    int i;
    /* 查表法, 避免复杂的 ASCII 码计算 */
    static const char t[] = "0123456789abcdef";

    /* 64bit 数字共包含 16 个十六进制字符 (4 bit / 字符)。
     * 从最高位 (15) 开始向低位剥离打印, 符合阅读习惯 */
    for (i = 15; i >= 0; i--)
        exc_putc(t[(v >> (i * 4)) & 0xF]); // 移位并使用 0xF 掩码提取当前半字节
}
```

4. 调试闭环流程建议

1. **观察现象**：系统失去响应，诊断串口（UART2）输出 [EXC] sp=... elr=... esr=... far=... 并锁死 CPU。
2. **初步定性**：观察 esr 判断异常类型，观察 far 确认是否为常见的内存越界。
3. **定位代码**：在宿主机 Ubuntu 终端，使用交叉编译工具链：

```
aarch64-linux-gnu-addr2line -e core1_firmware.elf <elr_hex_value>
```

4. **排查上下文**：结合定位到的代码行，检查指针合法性、跨核时序约束（是否缺失 Memory Barrier）或 MMU 配置。